

Proyecto V8

19 de Abril del 2026



José Manuel Sánchez Rosal

-

*TFG Aplicación Web para Reserva de Citas
Desarrollo de Aplicaciones Multiplataforma
2026 IES Antonio Gala (Palma del Río)*

-

Índice

Índice.....	2
1- Introducción.....	3
2- Definición del Problema.....	4
2.1 Definición del Problema real.....	4
2.2 Problema Técnico.....	5
3. Objetivos.....	5
3.1 Objetivo Principal.....	5
3.2 Objetivos Específicos.....	6
4. Antecedentes.....	6
5. Recursos.....	8
5.1 Recursos Humanos y Conocimientos.....	8
5.2 Recursos Hardware.....	8
6. Análisis del Problema.....	8
6.1 Propuesta Software.....	8
6.1.1 Requisitos de Usuario.....	9
6.1.2 Requisitos Funcionales.....	10
6.1.3 Requisitos No Funcionales.....	11
7. Diseño de la solución software.....	12
7.1. Modelo relacional de la base de datos.....	12
7.2. Normalización de la base de datos.....	13
7.3. Paquetes o tecnologías.....	14
8. Diseño de la interfaz.....	15
8.1. Navegación, Encabezado y pié de Página.....	15
8.2. Catálogo de Servicios (Vista Cliente).....	16
8.3. Proceso de Reserva (Flujo Lineal).....	16
8.4. Paneles de Usuario (Perfiles).....	17
8.5. Adaptabilidad a Dispositivos (Checklist Mobile/PC).....	18
9. Pruebas.....	18
9.1. Test de Verificación de Infraestructura.....	18
9.2. Test de Gestión de Identidad y Seguridad.....	18
9.3. Test de Negocio de Aplicación.....	19
9.4. Documentos de entrada y salida.....	19

9.5 Herramientas del Entorno de Desarrollo.....	20
10. Conclusiones.....	21
10.1. Futuras Líneas de Mejora.....	21
11. Bibliografía.....	22
Documentación Oficial de Tecnologías.....	22
Libros y Manuales.....	22
Artículos y Recursos Web.....	23
Herramientas y Software Utilizado.....	23

Implementar:

VOY A VENDER ESTO, IMPLEMENTAR FUNCIONALIDADES QUE ME GUSTARÍA QUE TUVIERA LA APP

- 1- Notificación a los clientes de modificación o asignación de citas por parte del jefe.
- 2- Si el cliente cancela o modifica, que le notifique al jefe.
- 3- Pensar si modificar la visualización de la carga de los días de calendario.
- 5- Diferenciar clientes habituales de los casuales o nuevos
- 6- Implementar solución para dar citas a clientes no registrados.
- 4- Futura mejora: google calendar.

1- Introducción

La situación particular que motiva este proyecto es **la dificultad que enfrenta un profesional autónomo para encontrar herramientas de gestión que se ajusten a su realidad**. La mayoría del software existente en el mercado está diseñado para grandes salones con

múltiples empleados, resultando excesivamente complejo, costoso o con funcionalidades innecesarias para una gestión unipersonal.

Mi Proyecto Final Intermodular del **Ciclo Superior de Desarrollo de Aplicaciones Multiplataforma** consiste en el **desarrollo de una Aplicación Web de Gestión de Citas Online** para un Salón de Belleza y Peluquería unipersonal (autónoma).

El tiempo es Oro, por ello la aplicación busca digitalizar y automatizar el proceso de reserva, permitiendo a los clientes agendar citas 24/7 **liberando al profesional de la necesidad de atender llamadas o mensajes** constantes, optimizando su tiempo y flujo de trabajo diario (dentro o fuera de su jornada laboral).

2- Definición del Problema

Para abordar correctamente la solución, es necesario distinguir entre el problema operativo del negocio y los desafíos técnicos que conlleva su solución.

2.1 Definición del Problema real

El problema central es la ineficiencia en la gestión del tiempo y la comunicación del Profesional autónomo, lo que genera:

1. **Interrupciones constantes:** Las llamadas, los WhatsApps o mensajes directos vía redes sociales interrumpen continuamente el servicio al cliente presencial, disminuyendo la calidad de trabajo y la concentración.
2. **Gestión manual propensa a errores:** El uso de agendas o calendarios no automatizados conlleva riesgos de errores humanos, como dobles reservas (solapamiento), huecos perdidos por mala organización o citas olvidadas por falta de confirmación.
3. **Restricción horaria para el Cliente:** Actualmente, la captación de citas está limitada al momento en que el profesional puede responder, perdiendo oportunidades de reserva fuera del horario laboral o durante fines de semana.

2.2 Problema Técnico

Desde el **punto de vista del desarrollo de software**, la gestión manual actual **presenta las siguientes deficiencias técnicas**:

- **Ineficiencia en el cálculo de disponibilidad:** Solapamiento entre citas, la diferencia entre el tiempo estimado de duración del tratamiento y el tiempo real.
- **Falta de integridad y persistencia de datos:** Al no existir una base de datos centralizada, la información de los clientes al igual que su historial de citas está disperso y abierto a pérdidas físicas.
- **Falta de una plataforma centralizada de administración:** Actualmente, la gestión de los servicios ofrecidos, sus tarifas, la duración de cada tratamiento y los ajustes necesarios en la agenda dependen de métodos manuales (agenda física, notas, hojas sueltas, etc).
- **Ausencia de concurrencia:** El sistema actual no permite que múltiples usuarios consulten la disponibilidad al mismo tiempo sin riesgo de conflicto.
- **Solución mediante aplicación Multiplataforma.**
- **Realizar una Interfaz sencilla para solicitar cita o servicio en el salón de peluquería.**
- Impedir solicitar la misma cita (día y hora) por más de un cliente.

3. Objetivos

3.1 Objetivo Principal

Desarrollar una Aplicación Web funcional y robusta que automatice la reserva de citas, ajustando de manera dinámica la disponibilidad del calendario en función del tratamiento seleccionado, diseñada específicamente para las necesidades de una Profesional Autónoma.

Nombre App: "Tuturno"

3.2 Objetivos Específicos

1. **Automatización y Disponibilidad:** Permitir a los clientes la reserva, consulta, cancelación o modificación de citas a través de la aplicación **accesible las 24 horas del día**, 7 días a la semana, durante todo el año.
2. **Lógica de agenda Dinámica:** Implementar en el servidor la capacidad lógica de calcular y bloquear automáticamente tramos horarios correctos en el calendario, considerando la duración del tratamiento y los tiempos de preparación y limpieza necesarios.
3. **Gestión administrativa Centralizada:** Proporcionar un papel de administración seguro e intuitivo donde el profesional pueda gestionar sus servicios y tarifas sin necesidad de conocimientos técnicos avanzados (**CRUD servicios y citas bajo el Rol de Administrador**).
4. **Escalabilidad y mantenibilidad:** Desarrollar el sistema bajo una arquitectura de Software Profesional que asegure un código limpio, mantenible y escalables a futuras funcionalidades del negocio.
5. **Gestión de Usuarios:** la aplicación permitirá el alta y baja de clientes.

4. Antecedentes

El proyecto surge tras analizar el mercado de software comercial de gestión de salones y detectar una brecha entre la oferta y las necesidades reales de una microempresa.

Análisis de Soluciones Existentes: El mercado ofrece soluciones robustas y reconocidas como Booksy, Fresha o Koibox. A continuación se detallan algunas de sus características:

1. **Booksy:** Es una plataforma muy popular y completa que permite reservas online 24/7, gestión de agenda, clientes, pagos y marketing. Muy útil si quieres visibilidad (mercado de usuarios muy grande) y abarcar múltiples servicios en un solo lugar.

2. **Fresha:** Sistema de reservas orientado a salones de belleza y bienestar, con interfaz limpia y fácil de usar.
3. **Koibox:** Es un software de gestión integral para salones de peluquería y belleza, incluye agenda, gestión de clientes, inventario, informes. También herramientas de administración interna y reservas online

Ahora veremos los puntos débiles de los mismos, punto crucial para la decisión de que estas apps no encajan en el perfil del Autónomo Unipersonal:

- A. **Exceso de funcionalidades innecesarias:** todas incluyen módulos avanzados (stock, marketing, analíticas complejas, gestión multiempleo) que un profesional autónomo no necesita, haciendo la interfaz más compleja y menos intuitiva.
- B. **Costes recurrentes:** Muchos servicios requieren de una suscripción mensual, pagos adicionales o comisiones por reserva, lo cual incrementa los gastos fijos del autónomo.
- C. Dependencia de plataformas externas: Los datos, agenda y disponibilidad dependen de servidores y lógicas externas, limitando la personalización. Además, los clientes pueden ver otros negocios, creando competencia.
- D. **Falta total de personalización:** Estas apps presentan estructuras rígidas para configurar servicios, duración, descansos o disponibilidad.
- E. **Curva de aprendizaje innecesaria:** Su complejidad está pensada para equipos grandes, obliga al profesional a aprender funciones que nunca usará.

Por lo tanto, este proyecto busca **replicar en parte la funcionalidad central** de reserva de estas grandes plataformas pero en una solución hecha a medida, **gratuita** (sin costes de licencia) y **optimizada para** la agilidad de **un solo Usuario**.

5. Recursos

5.1 Recursos Humanos y Conocimientos

- **Coordinador del Proyecto:** Pedro Rafael García Velasco (Tutor y profesor de asignaturas clave).
- **Desarrollador Principal:** José Manuel Sánchez Rosal (Estudiante de Ciclo Superior DAM)
- **Asesor y colaborador:** José Julio Soldado Carmona (ex-alumno Ciclo Superior DAM)
- **Conocimientos Técnicos Requeridos:** para la ejecución del Proyecto se aplicarán conocimientos avanzados de desarrollo web Full Stack, seleccionando las herramientas más adecuadas durante la fase de análisis:
 - Backend: Desarrollo de API REST segura y robusta utilizando lenguajes de programación de servidor y frameworks de desarrollo ágil.
 - Persistencia: Diseño y gestión de Bases de Datos Relacionales y normalización de datos para el manejo eficiente de Usuarios, Servicios y Transacciones.
 - Frontend: Desarrollo de Interfaces Web dinámicas y responsivas y accesibles basadas en estándares web actuales.
 - DevOps: Control de versiones distribuidos y despliegue en entornos de servidor en la nube.

5.2 Recursos Hardware

- Portátil Medion Akoya Intel CORE i5 (8ª GEN) con 16 gb de RAM.
- Monitor Samsung de 27" resolución Full HD con panel PLS.

6. Análisis del Problema

Vamos a desglosar la definición de nuestro problema para sacar a luz nuestros requisitos:

6.1 Propuesta Software

Los requisitos que debe cumplir nuestra app, se dividen en tres grupos:

6.1.1 Requisitos de Usuario

Tenemos dos tipos de Rol de usuario, el **usuario normal** y el **usuario administrador**. A través de la página, en la pestaña "zona clientes", se podrá acceder al registro de usuarios mediante un formulario. A continuación se muestran los requisitos de Usuario de cada uno:

- Un Usuario debe poder registrarse.
- Usuario y Administrador podrán loguearse.
- Usuario y Administrador podrán acceder al calendario.
- El usuario podrá visualizar los días y huecos disponibles para reservar un servicio (las citas ocupadas serán anónimas, sin poder ver el cliente que la tiene ocupada).
- El administrador podrá ver los días y huecos disponibles para visualizar todas y cada una de las citas de sus clientes.
- El Usuario podrá seleccionar fecha y hora y servicio requerido para reservar una cita.
- Usuario y Administrador podrán modificar la cita.
- Usuario y Administrador podrán eliminar la cita.
- El Usuario podrá darse de baja como usuario de la página y dejará de tener acceso al calendario y servicios.
- El administrador podrá gestionar el catálogo de servicios, teniendo capacidad para crear nuevos tratamientos, modificar sus datos o eliminarlos.
- El usuario recibirá una confirmación visual de confirmación de reserva.
- El usuario podrá reportar errores o incidencias de la plataforma mediante un formulario estructurado desde su zona personal.
- El administrador de sistema (Superadmin) tendrá acceso a un registro centralizado de los errores reportados por los clientes.
- El administrador de sistema podrá impersonar (iniciar sesión como) cualquier usuario registrado para realizar comprobaciones en vivo y resolver incidencias.

6.1.2 Requisitos Funcionales

- Se podrá enviar un formulario con los datos del cliente al servidor para su registro.
- Cliente y administrador, al enviar sus datos de inicio de sesión al servidor a través de un formulario, recibirá ciertos permisos en la página .
- Cliente y administrador una vez logueados, podrán acceder al calendario de citas.
- En el calendario, el usuario podrá verificar los huecos disponibles.
- En el calendario, el administrador podrá visualizar las citas ocupadas por cada cliente.
- El usuario podrá seleccionar fecha, hora y servicio seleccionado para reservar su cita, la información será enviada a modo de formulario.
- Usuario y administrador podrán modificar citas, a través del formulario.
- Usuario y administrador podrán acceder al calendario y eliminar la cita a través del formulario.
- El usuario podrá enviar un formulario para darse de baja como cliente para ser eliminado.
- El administrador podrá dar de alta, editar y eliminar servicios a través de un formulario.
- Cálculo de tiempos, el sistema calculará la disponibilidad de huecos sumando automáticamente el tiempo de "limpieza y preparación".
- El sistema permitirá al cliente enviar un formulario modal de reporte de errores capturando el tipo de fallo, dispositivo y descripción, almacenándolo en la base de datos.
- El sistema proveerá una vista al administrador para listar, filtrar por severidad/categoría y eliminar los informes de error.
- El sistema implementará una función de cambio de contexto de sesión, permitiendo al administrador navegar con los credenciales y la vista de un cliente específico sin requerir su contraseña original.
- El usuario podrá visualizar, modificar los detalles (servicio, fecha u hora) y eliminar sus citas confirmadas directamente desde su perfil personal.
- **Módulo de Soporte Avanzado:** El sistema debe implementar la funcionalidad de "impersonación", permitiendo al administrador iniciar sesión con la identidad de

cualquier usuario sin requerir su contraseña, con el fin de replicar errores o resolver problemas específicos de una cuenta.

- **Registro centralizado de incidencias:** Se debe proveer una herramienta al administrador para registrar, clasificar (por categoría de error y dispositivo) y gestionar el estado (ej. Solucionado, Pendiente) de los problemas técnicos reportados por clientes o la jefa, permitiendo también su eliminación.

6.1.3 Requisitos No Funcionales

Adaptabilidad (Diseño Responsivo): La aplicación debe visualizarse y funcionar correctamente tanto en ordenadores como en dispositivos móviles, ya que los clientes reservarán principalmente desde el teléfono.

Integridad y persistencia de Datos (Cero Solapamientos): El sistema debe garantizar técnicamente que nunca se puedan guardar dos citas en el mismo tramo horario. Si dos usuarios intentan reservar el mismo hueco al mismo tiempo, uno debe recibir un error amigable. Por otro lado, el administrador tendrá la seguridad de tener a salvo la información a través de la bbdd.

Seguridad de Acceso (Autenticación): El acceso a la zona de administración y perfil de usuario debe requerir usuario y contraseña.

Facilidad de Uso (Usabilidad): El proceso de reserva debe ser lineal y sencillo: Seleccionar Servicio -> Seleccionar Fecha/Hora -> Confirmar. No debe requerir conocimientos técnicos por parte del cliente.

NOTA: Para garantizar esta usabilidad en clientes sin experiencia tecnológica, el sistema implementará una **Guía Interactiva (Onboarding)** de un solo uso en su primer inicio de sesión, que les enseñará visualmente cómo completar el flujo de reserva.

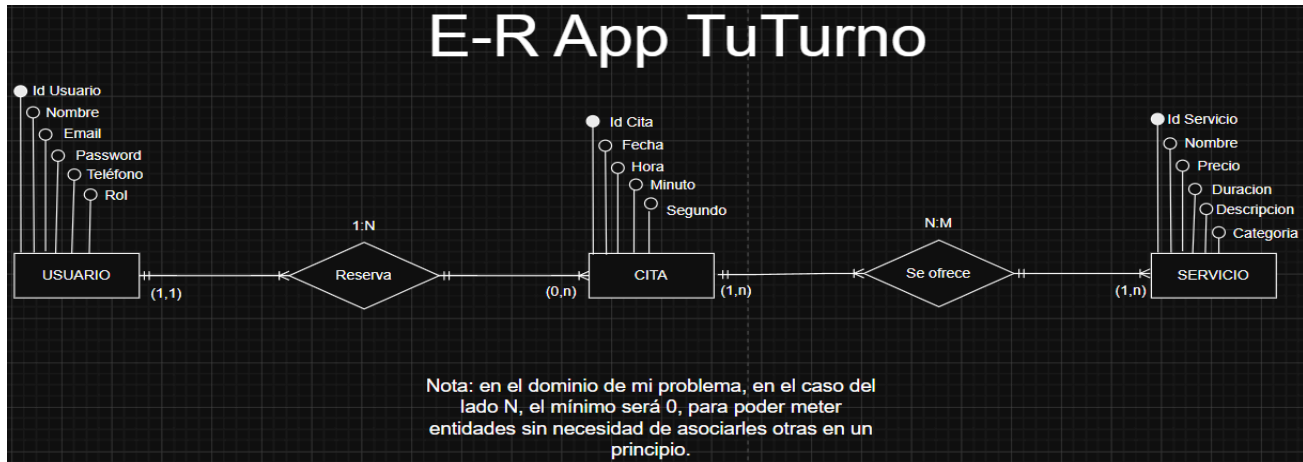
7. Diseño de la solución software

El sistema "Tuturno" se ha diseñado bajo una arquitectura cliente-servidor desacoplada, lo que permite una separación total entre la interfaz de usuario y la lógica de negocio. El **Backend** está construido como una API RESTful desarrollada en Java con el framework Spring Boot, siguiendo un patrón de diseño multicapa que separa controladores (exposición de servicios), lógica de negocio (capa de servicios) y persistencia (capa de repositorios). Por su parte, el **Frontend** se ha implementado con una interfaz web modular basada en estándares modernos (HTML5, CSS3 y Vanilla JavaScript), garantizando una comunicación asíncrona eficiente mediante peticiones HTTP.

7.1. Modelo relacional de la base de datos

El sistema utiliza PostgreSQL como base de datos relacional, gestionada mediante JPA/Hibernate. El esquema se compone de las siguientes tablas:

- **Tabla usuarios:** Almacena la información de acceso y contacto. Atributos: **id**(PK), **nombre**, **email**, **password**, **telefono** y **rol** (almacenado como String mediante el Enum **TipoRol**) y una bandera booleana ("ha visto guía") para gestionar el estado del tutorial interactivo.
- **Tabla servicios:** Contiene el catálogo de tratamientos. Atributos: **id**(PK), **nombre**, **descripcion**, **precio**, **duracion_minutos** y **categoria**.
- **Tabla citas:** Registra las reservas de tiempo. Atributos: **id**(PK), **fecha**, **hora**, **minutos**, **segundos** y **usuario_id**(FK).
- **Tabla intermedia Cita_Servicio:** Tabla necesaria para resolver la relación de muchos a muchos (@ManyToMany), vinculando múltiples servicios a una sola cita y viceversa. Atributos: **id_cita**(FK) e **id_servicio**(FK):
- **Tabla errores:** Almacena los reportes de incidencias de los clientes. Atributos: **id**(PK), **usuario_id**(FK), **tipo_error**, **dispositivo**, **descripcion**, **fecha**, **severidad** y **estado**.



7.2. Normalización de la base de datos

El diseño garantiza la integridad y eficiencia de los datos cumpliendo las tres primeras formas normales:

- **Primera Forma Normal (1FN):** Se garantiza que todos los atributos sean atómicos y no existan grupos repetitivos. El ejemplo principal es la tabla intermedia [Cita_Servicio](#), que desglosa la relación entre citas y tratamientos, evitando listas de valores en una sola columna.
- **Segunda Forma Normal (2FN):** Se cumple al estar en 1FN y asegurar que todos los atributos no clave dependan funcionalmente de forma completa de la clave primaria (PK). Dado que todas las tablas utilizan un [id](#) autoincremental como PK única, no existen dependencias parciales.
- **Tercera Forma Normal (3FN):** Se cumple al no existir dependencias transitivas entre atributos no clave. Por ejemplo, en la tabla [usuarios](#), el teléfono o el email dependen directamente del [id](#) del usuario y no de otros atributos no clave. Igualmente, los detalles de un servicio (precio, duración) dependen exclusivamente de su identificador único de servicio.

7.3. Paquetes o tecnologías

Para el desarrollo del proyecto "Tuturno", se han seleccionado las siguientes tecnologías, frameworks y entornos de desarrollo, organizados según su área de aplicación:

Backend (Servidor y Base de Datos)

- **Java:** Lenguaje de programación principal estructurado bajo el paradigma de orientación a objetos.
- **Spring Boot:** Framework principal utilizado para la creación de la API RESTful.
- **Spring Data JPA / Hibernate:** Herramientas de mapeo objeto-relacional (ORM) para la comunicación y persistencia de datos con la base de datos.
- **Lombok:** Librería utilizada para reducir el código repetitivo en las clases Java (como *getters*, *setters* y constructores). * **PostgreSQL:** Sistema de gestión de bases de datos relacional para almacenar la información de usuarios, citas y servicios. * **IntelliJ IDEA:** Entorno de Desarrollo Integrado (IDE) principal utilizado para la programación de la lógica de servidor.

Frontend (Interfaz de Usuario) * HTML5 y CSS3:

Lenguajes estándar para la estructura semántica y el diseño visual y responsivo de la aplicación. * **Vanilla JavaScript:** Utilizado para la lógica del lado del cliente, manipulación del DOM y peticiones asíncronas HTTP a la API, sin depender de frameworks adicionales. * **Visual Studio Code:** Editor de código ligero utilizado para el desarrollo de la interfaz web.

Pruebas (Testing)

- **Postman:** Plataforma y cliente API utilizado de forma intensiva para diseñar, probar y validar el correcto funcionamiento de los distintos *endpoints* del Backend antes de su integración con el Frontend.
- **Control de Versiones** * **Git:** Sistema de control de versiones distribuido para el seguimiento de los cambios en el código fuente. * **GitHub:** Plataforma en la nube utilizada para alojar el repositorio central del proyecto y respaldar el código.

8. Diseño de la interfaz

Introducción y Principios de Diseño

El diseño de la interfaz (UI) de "Tuturno" se ha centrado en dos pilares fundamentales: la sencillez y la adaptabilidad. El objetivo es que el cliente pueda realizar una reserva en menos de un minuto, sin necesidad de formación previa ni ayuda técnica. Siguiendo los principios de usabilidad definidos en los requisitos no funcionales, el diseño es **completamente adaptativo (Responsive Design)**. Esto garantiza una experiencia de usuario (UX) óptima e intuitiva en cualquier dispositivo: **ordenadores personales (PC), tablets y teléfonos móviles**.

8.1. Navegación, Encabezado y pie de Página

El encabezado (Header) se mantiene fijo en la parte superior para proporcionar acceso rápido a la navegación. Incluye:

- **Logotipo de Marca:** Refuerza la identidad del salón de belleza.
- **Menú de Navegación Principal:** En resoluciones de PC y tablet, se muestran enlaces directos como "Inicio", "Servicios", "Zona Clientes" y "Contacto". En dispositivos móviles, este menú se contrae dentro de un icono de "hamburguesa" (☰) para maximizar el espacio de contenido.
- **Botón de Acción:** Un acceso directo visible para "Reservar Cita" o "Perfil/Salir" (según el estado de login).
- **Botón de Ayuda Interactiva:** Integrado de forma accesible en el menú inferior derecho, permitiendo al usuario volver a lanzar el tutorial visual (`Guía Interactiva`) en cualquier momento si tiene dudas durante el proceso.
- **Pie de página:** Como elemento de cierre transversal a todas las vistas de la aplicación, se ha implementado un pie de página (*footer*) dinámico y responsivo. Su objetivo principal es actuar como un repositorio centralizado de la información corporativa, legal y de contacto del salón de belleza, garantizando que el cliente siempre tenga a su disposición vías de comunicación alternativas a la reserva online.

8.2. Catálogo de Servicios (Vista Cliente)

Diseñado como una galería visual, el catálogo presenta los servicios organizados por tarjetas. Cada tarjeta incluye:

- **Icono Descriptivo:** Facilita la identificación visual del tratamiento.
- **Título y Descripción Breve:** Explica el servicio.
- **Detalles Clave:** Duración estimada (en minutos) y Precio, mapeados directamente desde la entidad [Servicio](#).
- **Selector de Reserva:** Un checkbox que permite al usuario añadir el servicio a su cita.

8.3. Proceso de Reserva (Flujo Lineal)

El flujo de reserva se ha simplificado al máximo para garantizar la usabilidad:

1. **Selección:** El usuario elige sus servicios del catálogo.
2. **Calendario Dinámico:** Se muestra un calendario interactivo que, basándose en la lógica del backend ([Servicio](#), [Cita](#) y tiempos de limpieza), bloquea automáticamente los huecos no disponibles. El usuario selecciona día y hora.
3. **Confirmación:** Una pantalla de resumen permite verificar la cita (fecha, hora, servicios) antes de enviarla. Una vez confirmada, el usuario ve una notificación visual de éxito.
4. **Guía Interactiva de reserva:** Se ha desarrollado un sistema de superposiciones dinámicas (overlays) que oscurece la interfaz general pero ilumina secuencialmente los elementos clave del DOM mediante un efecto 'Spotlight'. Esta guía acompaña al cliente por los pasos descritos anteriormente (Selección , Calendario y Confirmación), finalizando con un claro 'Call to Action' sobre el botón de reserva. Esto previene el abandono de la aplicación por desconocimiento del sistema.

8.4. Paneles de Usuario (Perfiles)

- **Zona Clientes:** Espacio personal donde el usuario gestiona su perfil y visualiza sus próximas citas. Incluye una herramienta de soporte directo mediante un botón de "Reportar un error", que despliega un formulario modal optimizado para identificar fallos de visualización o lógica de reserva.
- **Panel de Jefatura (Agenda y Servicios):** Este panel operativo centraliza la gestión diaria del salón.
 - **Agenda Interactiva:** Presenta una vista de calendario donde los días con citas confirmadas se distinguen visualmente (por ejemplo, en verde). Al seleccionar un día, el profesional puede visualizar una lista detallada de las reservas, mostrando el nombre del cliente, la hora exacta y el tratamiento solicitado. Desde esta vista se habilitan las funciones de Crear, Modificar y Borrar citas de forma manual para gestionar imprevistos o reservas telefónicas.
 - **Mantenimiento de Catálogo:** En una pestaña independiente, el usuario tiene acceso al CRUD de servicios, permitiéndole crear nuevos tratamientos, editar sus detalles (nombre, precio, duración técnica, tiempo de limpieza) o borrarlos del catálogo.
- **Panel de Administración del Sistema:** Herramienta técnica orientada al mantenimiento y soporte de la plataforma.
 - **Gestión Integral de Usuarios:** Permite visualizar el listado completo de usuarios registrados y ejecutar acciones de CRUD (Crear, Leer, Actualizar, Borrar). Incluye la función de acceso por impersonación para diagnósticos avanzados.
 - **Gestión de Incidencias Técnicas:** En una pestaña independiente, el administrador puede gestionar los reportes de fallos del sistema enviados por los usuarios. Esta herramienta permite clasificar los errores por categoría de

fallo y dispositivo de acceso, marcar incidencias como solucionadas y borrarlas una vez procesadas.

8.5. Adaptabilidad a Dispositivos (Checklist Mobile/PC)

El diseño se adapta automáticamente de la siguiente manera:

- **PC/Tablet:** El diseño se expande horizontalmente, aprovechando el ancho de pantalla.
- **Móvil:** Las tarjetas de servicios se compactan y el menú se oculta para una navegación táctil fluida.

9. Pruebas

El apartado de pruebas del proyecto certifica la solidez técnica de la aplicación "TuTurno" mediante la validación automatizada de su infraestructura y lógica de negocio principal. A través de diferentes suites, se audita el correcto arranque del servidor, la seguridad estricta en el cifrado de contraseñas y la gestión integral de identidades. Además, se somete a estrés el núcleo de la aplicación, verificando minuciosamente el sistema de permisos por roles, la disponibilidad de los horarios y la prevención de solapamientos en las citas.

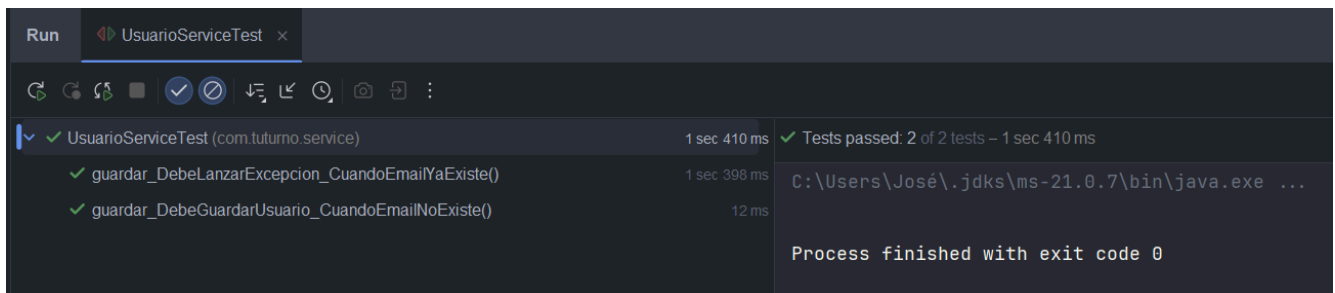
9.1. Test de Verificación de Infraestructura

Verifica que el contexto de Spring Boot y la inyección de dependencias se inicializan correctamente sin errores críticos de arranque:

```
✓ TuturnoApplicationTests (com.tuturno) 31 ms ✓ Tests passed: 1 of 1 test – 31 ms
  ✓ contextLoads() 31 ms C:\Users\José\.jdk\ms-21.0.7\bin\java.exe ...
  Process finished with exit code 0
```

9.2. Test de Gestión de Identidad y Seguridad

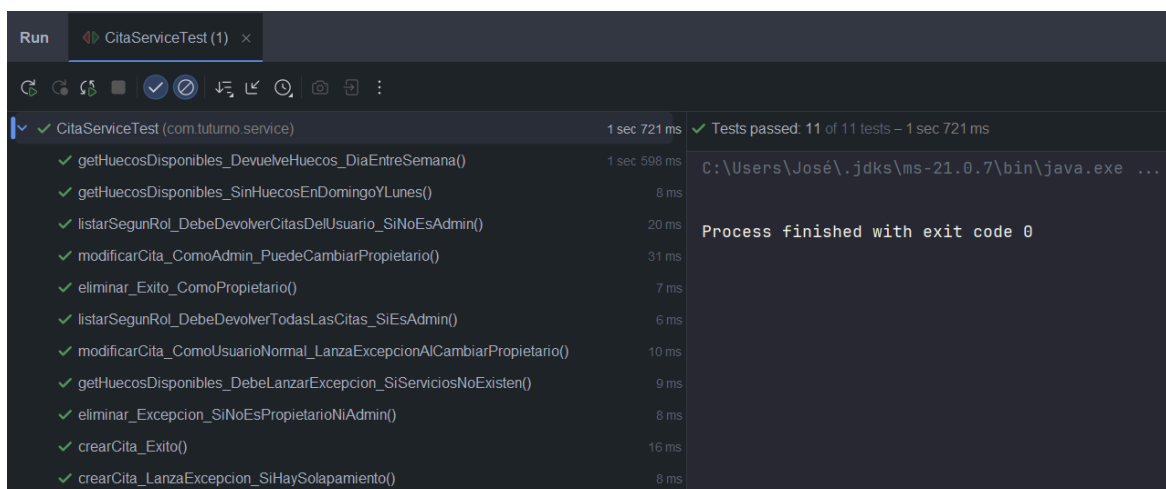
Valida que las contraseñas se encriptan (hash) obligatoriamente antes de almacenarse y bloquea el registro si el correo electrónico ya existe:



```
Run  UsuarioServiceTest x
✓ UsuarioServiceTest (com.tuturno.service) 1 sec 410 ms ✓ Tests passed: 2 of 2 tests – 1 sec 410 ms
  ✓ guardar_DebeLanzarExcepcion_CuandoEmailYaExiste() 1 sec 398 ms C:\Users\José\.jdk\ms-21.0.7\bin\java.exe ...
  ✓ guardar_DebeGuardarUsuario_CuandoEmailNoExiste() 12 ms
Process finished with exit code 0
```

9.3. Test de Negocio de Aplicación

Audita el control de permisos según el rol (Admin vs. Usuario común), el cálculo matemático de huecos disponibles (respetando días inhábiles y tiempos de preparación) y el bloqueo estricto de reservas para evitar solapamientos:



```
Run  CitaServiceTest (1) x
✓ CitaServiceTest (com.tuturno.service) 1 sec 721 ms ✓ Tests passed: 11 of 11 tests – 1 sec 721 ms
  ✓ getHuecosDisponibles_DevuelveHuecos_DiaEntreSemana() 1 sec 598 ms C:\Users\José\.jdk\ms-21.0.7\bin\java.exe ...
  ✓ getHuecosDisponibles_SinHuecosEnDomingoYLunes() 8 ms
  ✓ listarSegunRol_DebeDevolverCitasDelUsuario_SiNoEsAdmin() 20 ms
  ✓ modificarCita_ComoAdmin_PuedeCambiarPropietario() 31 ms
  ✓ eliminar_Exito_ComoPropietario() 7 ms
  ✓ listarSegunRol_DebeDevolverTodasLasCitas_SiEsAdmin() 6 ms
  ✓ modificarCita_ComoUsuarioNormal_LanzaExcepcionAlCambiarPropietario() 10 ms
  ✓ getHuecosDisponibles_DebeLanzarExcepcion_SiServiciosNoExisten() 9 ms
  ✓ eliminar_Excepcion_SiNoEsPropietarioNiAdmin() 8 ms
  ✓ crearCita_Exito() 16 ms
  ✓ crearCita_LanzaExcepcion_SiHaySolapamiento() 8 ms
Process finished with exit code 0
```

9.4. Documentos de entrada y salida

A continuación, se detalla el diseño de los principales casos de validación del sistema, mostrando las entradas inyectadas y las salidas obtenidas durante la ejecución de los tests unitarios:

ID Prueba	Descripción	Datos de Entrada (Input)	Salida Esperada (Expected Output)	Resultado
CP-01	Cálculo de huecos en día laboral	Fecha: Jueves laborable Servicio: 45 minutos	Lista de franjas horarias con tamaño 45 mins separadas por tiempos de limpieza.	✓ Éxito (Build Success)
CP-02	Cálculo de huecos en día de cierre	Fecha: Lunes / Domingo Servicio: Cualquier duración	Lista vacía [] (Sin disponibilidad).	✓ Éxito (Build Success)
CP-03	Crear cita correctamente	Hora: 10:00 - 10:45 Estado: Sin solapes	Cita creada exitosamente y guardada en el simulador de Base de Datos.	✓ Éxito (Build Success)
CP-04	Prevención de Solapamiento	Hora: 10:00 - 10:45 Estado previo: Ya hay una cita a las 10:15	Error del sistema: <code>IllegalArgumentException</code> ("El servicio se solapa con una cita existente").	✓ Éxito (Build Success)
CP-05	Fallo por manipulación de ID	Usuario A intenta modificar una cita que pertenece a Usuario B.	Error del sistema: <code>SecurityException</code> o Acceso Denegado (403 Forbidden).	✓ Éxito (Build Success)
CP-06	Acceso Administrativo	Usuario con rol ADMIN intenta modificar una cita del Usuario B.	Cita modificada exitosamente (Privilegio superior aplicado).	✓ Éxito (Build Success)

- Conclusión de los ensayos: Como se observa en la matriz de pruebas, el flujo de información de entrada de los usuarios pasa por los filtros de seguridad y viabilidad horaria, respondiendo el sistema de forma determinista para cada valor límite probado e impidiendo corrupciones en el calendario de reservas.

9.5 Herramientas del Entorno de Desarrollo

Para optimizar la fase de desarrollo y la depuración conjunta de frontend y backend, se implementó de forma temporal un panel de 'Acceso Rápido (Dev)' en la pantalla de autenticación. Este mecanismo permite el acceso instantáneo al sistema simulando los tres roles principales (Cliente, Jefe y Admin), lo que facilita enormemente la ejecución de pruebas manuales y la validación ágil del control de accesos sin la necesidad de introducir credenciales de prueba de forma manual y repetitiva.

10. Conclusiones

Resumen de Logros y Solución de Problemas El desarrollo del proyecto "Tuturno" ha concluido con éxito, cumpliendo con el objetivo principal de crear una Aplicación Web robusta y funcional que automatiza la gestión de citas online para un salón de belleza unipersonal. La solución implementada resuelve eficazmente el problema operativo de las interrupciones constantes, la gestión manual propensa a errores y la restricción horaria, proporcionando una herramienta profesional que digitaliza y optimiza el flujo de trabajo de un autónomo.

Solidez Técnica y Escalabilidad Técnicamente, la arquitectura cliente-servidor desacoplada (Backend API REST en Spring Boot / Frontend Vanilla JS) ha demostrado ser una elección acertada, garantizando un código limpio, mantenible y escalable. El esquema relacional de la base de datos, totalmente normalizado (1FN, 2FN, 3FN) como se detalla en el Apartado 7, asegura la integridad y persistencia de los datos, eliminando riesgos de redundancia y solapamientos de citas.

Experiencia de Usuario y Adaptabilidad La interfaz, diseñada bajo principios de usabilidad y adaptabilidad, ofrece una experiencia de usuario excelente tanto para el cliente como para el administrador. El diseño completamente responsive asegura la accesibilidad total desde **teléfonos móviles, tablets y PC**, cumpliendo con la premisa de que los clientes reservarán principalmente desde el teléfono.

10.1. Futuras Líneas de Mejora

- **Acceso directo mediante Código QR:** Tras el despliegue del sistema en un entorno de producción real, se generará un código QR físico para colocar en el mostrador del salón. Esto permitirá a los clientes presenciales escanearlo e instalar la aplicación en sus dispositivos móviles de forma instantánea, fomentando la transición hacia la reserva digital.
- **Integración de pasarela de pago:** Se añadirá un sistema de cobro online seguro dentro del flujo de reserva de la aplicación. Esta funcionalidad permitirá solicitar el pago de una

fianza o el importe total por adelantado, garantizando el compromiso de asistencia del cliente y reduciendo significativamente las citas fallidas (no-shows).

- **Módulo de contabilidad automatizada:** Se desarrollará una nueva sección en el panel de administración enfocada en la gestión financiera del negocio. Este módulo registrará los ingresos de los servicios prestados y generará informes o resúmenes periódicos, facilitando al profesional autónomo el control de su facturación y la preparación de impuestos.
- **Sistema de Alertas Críticas:** Implementación de un servicio de notificaciones automáticas (vía email o Webhook) que avise al administrador en el instante en que un usuario reporte un error de severidad "Crítica", permitiendo una respuesta inmediata ante caídas del servidor o fallos en la pasarela de pagos.

11. Bibliografía

Nota: La bibliografía citada a continuación corresponde a la documentación de las tecnologías candidatas en la fase de propuesta inicial. Estas referencias podrán actualizarse:

Documentación Oficial de Tecnologías

- Oracle. *Java SE Documentation*. Oracle, 2025. <https://docs.oracle.com/en/java/>
- Spring. *Spring Boot Reference Documentation*. Pivotal, 2025. <https://spring.io/projects/spring-boot>
- PostgreSQL Global Development Group. *PostgreSQL Documentation*. 2025. <https://www.postgresql.org/docs/>
- Mozilla Developer Network. *HTML, CSS y JavaScript*. Mozilla, 2025. <https://developer.mozilla.org>

Libros y Manuales

- Deitel, P., & Deitel, H. *Java: How to Program*. Pearson, 2023.
- Freeman, E., & Robson, E. *Head First HTML and CSS*. O'Reilly, 2020.

Artículos y Recursos Web

- Booksy. *Gestión de reservas online*. Booksy, 2025. <https://www.booksy.com/es>
- Fresha. *Software de gestión de salones*. Fresha, 2025. <https://www.fresha.com>
- Koibox. *Plataforma de reservas para peluquerías*. Koibox, 2025. <https://www.koibox.com>

Herramientas y Software Utilizado

- IntelliJ IDEA. JetBrains, 2025. <https://www.jetbrains.com/idea/>
- Visual Studio Code. Microsoft, 2025. <https://code.visualstudio.com/>
- Git. Software de control de versiones, 2025. <https://git-scm.com/>
- GitHub. Plataforma de repositorios, 2025. <https://github.com/>
- **ChatGPT**. OpenAI, 2025. <https://openai.com/chatgpt>
- **Gemini**. Google, 2025. <https://gemini.google.com/>